



Sveučilište J. J. Strossmayera

Fakultet elektrotehnike, računarstva i
informatičkih tehnologija Osijek

Cara Hadrijana 10b

HR-31000 Osijek

www.ferit.unios.hr

Laboratorijska vježba 1:
Osnovni MPI program

Ak. godina 2024/2025.

In Parallel, Everyone Hears You Scream (part deux)

A guy walks into a breakfast joint with 16 penguins. They sit down at the biggest table in the place. The guy orders coffee for himself and a bowl of cereal for each of the penguins. He then breaks out a newspaper and casually starts reading. Meanwhile, the penguins' breakfasts arrive and the first penguin starts eating while all the others look at him. After he finishes, all the penguins look at the second penguin while he eats. When the last penguin finishes his cereal, he emits a loud "gwank!" and all the penguins get up and file out of the restaurant.

The guy looks up, folds up his newspaper, and gets up to pay the bill. One of the other patrons had been watching the spectacle said, "Excuse me sir, I have to ask. What was that all about? Why did you just sit there while your penguins ate their breakfast?"

"Yeah, it always takes this long," he said. "It's cerealized."

Inicijalizacija i osnovni MPI program

1 Uvod u MPI (ažurirano za MPI-4.0)

MPI (*engl. Message Passing Interface – Sučelje za razmjenu poruka*) standard je za paralelno računarstvo koji implementira model razmjene poruka. Omogućava podatkovnu komunikaciju između procesa koristeći funkcije i potprograme dostupne u jezicima kao što su C, C++, i Fortran. Iako MPI programi koriste bibliotečne pozive za razmjenu poruka, sam MPI nije biblioteka, već specifikacija koja definira pravila za razmjenu poruka u paralelnim programima.

Cilj MPI programa je izvođenje dva ili više samostalnih procesa, pri čemu svaki proces izvršava svoj kod koji može, ali ne mora, biti identičan kodu ostalih procesa. Procesi komuniciraju putem MPI funkcija i identificiraju se prema **rangu**, jedinstvenom broju unutar grupe procesa.

Numeriranje ranga procesa je u obliku:

rang (0)	rang (1)	rang (2)	...	rang (veličina grupe procesa - 1)
----------	----------	----------	-----	-----------------------------------

Broj procesa definira se pri pokretanju programa koristeći naredbu **mpirun** ili **mpiexec** (ovisno o MPI implementaciji):

```
[zdravko@klaster ~]$ mpirun -np 4 ./ImeIzvrnogPrograma
```

```
[zdravko@klaster ~]$ mpiexec -n 4 ./ImeIzvrnogPrograma
```

1.1 Ključne komponente MPI standarda

MPI standard uključuje:

- **Specifikaciju funkcija i potprograma** za jezike C, C++, i Fortran, osiguravajući prenosivost programa. MPI programi bi trebali biti pokrenuti na bilo kojoj platformi koja podržava MPI standard.
- **Implementacijsku fleksibilnost**, gdje proizvođači mogu prilagoditi MPI za svoje platforme radi optimizacije performansi.
- **Podršku za više platformi**, uključujući klastere, superračunala i heterogene sustave.

1.2 Evolucija MPI standarda

1. MPI-2.0:

- Paralelni I/O
- Jednostrana komunikacija
- Dinamička kontrola broja procesa
- Prilagodbe za C++, Fortran 90

2. MPI-3.0:

- Neblokirajuća kolektivna komunikacija
- Poboljšana jednostrana komunikacija
- Novi modeli topologije procesa

3. MPI-4.0:

- Napredne optimizacije za heterogene sustave
- Proširenja kolektivnih operacija

Napomena: Iako je MPI-4.0 najnoviji standard, neke implementacije još uvijek djelomično ili uopće ne podržavaju MPI-3.0 ili MPI-4.0 funkcionalnosti.

1.3 Kategorije MPI rutina

MPI standard uključuje rutine za sljedeće operacije:

- **Komunikacija od točke do točke** (*point-to-point communication*)
- **Kolektivna komunikacija** (*collective communication*)
- **Jednostrana komunikacija** (*one-sided communication*)
- **Grupe procesa i komunikatori**
- **Topologije procesa**
- **Upravljanje paralelnim okruženjem i zahtjevima**

Moderni MPI alati, poput OpenMPI-ja ili MPICH-a, koriste **mpirun** ili **mpiexec** za pokretanje MPI programa. Primjer naredbe:

```
[zdravko@klaster ~]$ mpiexec -n 4 ./ImeIzvrnogPrograma
```

Potrebno je osigurati da je MPI okruženje pravilno postavljeno i da svi čvorovi imaju pristup programu te odgovarajućim MPI bibliotekama.

1.4 Praktični savjeti

- Preporuča se korištenje novijih verzija MPI alata radi bolje podrške za MPI-4.0.
- Preporuča se korištenje neblokirajućih i kolektivnih operacija za poboljšanje performansi u aplikacijama.
- Prilikom razvoja MPI programa, uvijek je potrebno testirati kod na različitim veličinama procesa kako bi se provjerila skalabilnost.

U sljedećim dijelovima bit će obrađene osnovne funkcije MPI-ja i prikazani primjeri koda za inicijalizaciju i osnovnu komunikaciju između procesa.

2 Struktura osnovnog MPI programa

Svi MPI programi slijede istu osnovnu strukturu:

1. Uključivanje MPI zaglavlja:

- Datoteka zaglavlja (mpi.h za C i C++, mpif.h za Fortran) sadrži potrebne definicije i prototipe funkcija specifičnih za MPI.

Uključivanje u C/C++:

```
#include <mpi.h>
```

Uključivanje u Fortranu:

```
include 'mpif.h'
```

2. Inicijalizacija MPI okruženja:

- Prvi korak svakog MPI programa je poziv rutine za inicijalizaciju okruženja:

```
MPI_Init(&argc, &argv);
```

3. Izvršavanje programskog koda:

- Nakon inicijalizacije, može se koristiti kombinacija standardnog koda i MPI rutina za komunikaciju i sinkronizaciju između procesa.

4. Završavanje MPI okruženja:

- Na kraju programa svaki proces mora pozvati rutinu za zatvaranje MPI okruženja:

```
MPI_Finalize();
```

5. Ako nijedan proces ne pozove ovu rutinu, program će se blokirati.

2.1 Standardi nazivlja za MPI rutine

C:

- Prefiks MPI_: Sva imena funkcija, konstanti i tipova započinju prefiksom MPI_ radi izbjegavanja konflikata s programskim varijablama.
- Imena rutina: Funkcije koriste velika i mala slova, npr. MPI_Comm_rank.
- Imena konstanti: Konstantne vrijednosti su pisane velikim slovima, npr. MPI_COMM_WORLD.

Povratne vrijednosti MPI funkcija

Većina MPI funkcija vraća cjelobrojni kod izlaza (exit code), koji pokazuje je li operacija uspješno izvršena:

- MPI_SUCCESS: Rutina je izvršena bez grešaka.
- Ostale vrijednosti: Predstavljaju specifične greške, ovisno o implementaciji.

Primjer provjere uspješnosti:

```
int err = MPI_Comm_rank(MPI_COMM_WORLD, &rank);
if (err != MPI_SUCCESS) {
    printf("Greška pri dohvaćanju ranga procesa\n");
}
```

2.2 MPI rukovoditelj

MPI održava interne podatkovne strukture koje se referenciraju kroz rukovoditelje (engl. *handles*). Ovi rukovoditelji predstavljaju apstrakcije nad objektima kao što su komunikatori, tipovi podataka i statusi. U jezicima C i C++, rukovoditelji su implementirani kao pokazivači ili cjelobrojne vrijednosti koje vraćaju razni MPI pozivi i mogu se koristiti kao argumenti u ostalim MPI funkcijama.

Primjeri rukovoditelja:

- **MPI_SUCCESS** – Cjelobrojni tip za testiranje izlaznih kodova.
- **MPI_COMM_WORLD** – Objekt tipa MPI_Comm koji predstavlja komunikator svih procesa.

Rukovoditelji su definirani u zaglavlju mpi.h i mogu se kopirati standardnim operatorima, poput =:

```
MPI_Comm custom_comm = MPI_COMM_WORLD;
```

2.3 Tipovi podataka MPI biblioteke

MPI pruža apstrakcije za tipove podataka koje omogućuju kompatibilnost između raznorodnih računalnih sustava (engl. *heterogeneous systems*). Prilikom korištenja MPI rutina, korisnik specificira MPI tipove podataka koji odgovaraju standardnim tipovima u jeziku C ili C++.

Tablica 1. Osnovni tipovi podataka MPI okruženja:

Tip podatka MPI biblioteke	C/C++ tip podatka
MPI_CHAR	signed char
MPI_UNSIGNED_CHAR	unsigned char
MPI_SIGNED_CHAR	signed char
MPI_SHORT	short
MPI_INT	int

MPI_LONG	long
MPI_UNSIGNED_SHORT	unsigned short
MPI_UNSIGNED	unsigned
MPI_UNSIGNED_LONG	unsigned long
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_LONG_LONG	long long
MPI_UNSIGNED_LONG_LONG	unsigned long long
MPI_WCHAR	wchar_t
MPI_BYTE	-
MPI_PACKED	-

2.3.1 Izvedeni tipovi podataka

MPI omogućuje definiranje vlastitih (izvedenih) tipova podataka koji mogu uključivati strukture ili polja podataka:

```
MPI_Datatype custom_type;
MPI_Type_contiguous(3, MPI_INT, &custom_type);
MPI_Type_commit(&custom_type);
```

Posebni tipovi podataka MPI biblioteke

MPI uvodi dodatne tipove podataka za specifične potrebe:

- **MPI_Comm** – Komunikatori za razmjenu poruka.
- **MPI_Status** – Informacije o statusu MPI operacija.
- **MPI_Datatype** – Korisnički definirani tipovi podataka.

Primjer deklaracije u C-u:

```
MPI_Status neki_status;
```

2.4 Inicijalizacija MPI okruženja

Svaki MPI program započinje pozivom funkcije `MPI_Init`, koja inicijalizira MPI okruženje. Ova rutina mora biti prva koja se poziva i može se pozvati samo jednom:

```
int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    // Ovdje ide glavni dio programa
    MPI_Finalize();
    return 0;
}
```

Argumenti:

- `argc` – Broj argumenata naredbenog retka.
- `argv` – Polje pokazivača na argumente naredbenog retka.

2.5 Isključivanje MPI okruženja

Na kraju programa, svaki proces mora pozvati `MPI_Finalize`, koja čisti interne strukture i završava sve aktivne operacije. Nakon poziva ove rutine, više nije moguće koristiti bilo koju MPI funkciju.

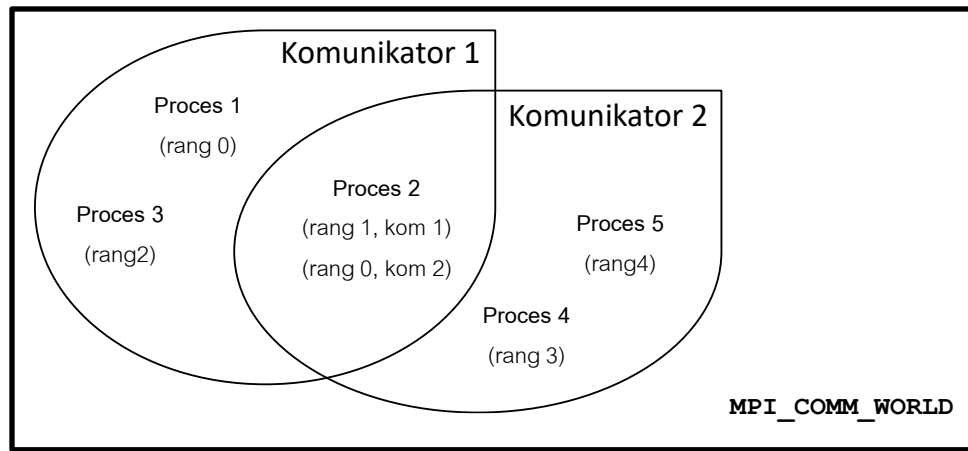
Primjer:

```
MPI_Finalize();
```

3 Komunikatori

Komunikator je MPI rukovoditelj koji definira grupu procesa s dozvoljenom međusobnom komunikacijom. Svaka MPI poruka mora specificirati komunikator pomoću imena, uključenog kao parametar u MPI funkciji. Komunikatori korišteni u pozivima za slanje i primanje poruka moraju međusobno odgovarati kako bi se uspostavila komunikacija.

MPI automatski pruža osnovni komunikator pod nazivom **MPI_COMM_WORLD**, koji uključuje sve procese pokrenute s programom. Koristeći **MPI_COMM_WORLD**, procesi mogu međusobno komunicirati bez dodatnog definiranja. Međutim, korisnici mogu definirati prilagođene komunikatore kako bi podijelili procese u manje grupe i omogućili specifične komunikacije unutar tih grupa.



Slika 1. Primjer komunikatora

3.1 Izrada prilagođenog komunikatora

Izrada novog komunikatora uključuje korištenje funkcije kao što je **MPI_Comm_split()**. Na primjer, procesi se mogu podijeliti u dvije grupe na temelju parnih i neparnih rangova.

Primjer u C-u:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Stvaranje boje (color) za podjelu na grupe
    int color = rank % 2; // Parni i neparni procesi
    MPI_Comm new_comm;
    MPI_Comm_split(MPI_COMM_WORLD, color, rank, &new_comm);

    int new_rank, new_size;
    MPI_Comm_rank(new_comm, &new_rank);
    MPI_Comm_size(new_comm, &new_size);

    printf("Rank %d u MPI_COMM_WORLD, rank %d u novom komunikatoru\n", rank, new_rank);

    MPI_Comm_free(&new_comm);
    MPI_Finalize();
    return 0;
}
```

Ovaj kod stvara dva komunikatora: jedan za procese s parnim rangovima i drugi za procese s neparnim rangovima.

3.2 Dohvat informacija o komunikatoru: rang i veličina

Dohvat ranga procesa

Proces može dobiti svoj rang unutar komunikatora pomoću funkcije **MPI_Comm_rank()**. Rangovi su jedinstveni unutar komunikatora i počinju od 0.

Dohvat veličine komunikatora

Ukupan broj procesa unutar komunikatora može se dohvatiti funkcijom **MPI_Comm_size()**. Ove funkcije omogućuju procesima da prilagode svoje ponašanje na temelju pozicije unutar komunikatora.

Primjer:

```
int rank, size;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
printf("Rank: %d, Veličina komunikatora: %d\n", rank, size);
```


3.3 Prednosti prilagođenih komunikatora

Korištenje prilagođenih komunikatora omogućuje:

- Organizaciju procesa u podgrupe za specifične zadatke.
- Povećanu čitljivost i modularnost koda.
- Izbjegavanje kolizija u komunikacijama između različitih dijelova programa.

3.4 Moderni pristup u C++

C++ korisnici mogu koristiti dodatne biblioteke poput Boost.MPI za jednostavniji i idiomatski rad s komunikatorima:

Primjer s Boost.MPI:

```
#include <boost/mpi.hpp>
#include <iostream>

namespace mpi = boost::mpi;

int main(int argc, char** argv) {
    mpi::environment env;
    mpi::communicator world;

    int rank = world.rank();
    int size = world.size();

    std::cout << "Rank: " << rank << ", Veličina komunikatora: " << size << std::endl;

    return 0;
}
```

Boost.MPI pojednostavljuje rad s MPI-jem pružajući sučelje prilagođeno C++ standardima, što omogućuje bolju integraciju s ostalim C++ značajkama poput STL-a.

3.5 Napomena o performansama

Prilikom korištenja prilagođenih komunikatora, važno je paziti na efikasnost i izbjegavati česta kreiranja i uništavanja komunikatora unutar petlji ili kritičnih dijelova koda. Umjesto toga, preporuča se kreirati komunikatore unaprijed i ponovno ih koristiti tijekom izvršenja programa.

3.6 Dohvat informacija o komunikatoru: *rang*

Proces može odrediti svoj rang u komunikatoru pozivom funkcije `MPI_Comm_rank()`. Kada se spominju rangovi procesa, bitno je napomenuti da:

- Rangovi su slijedni brojevi i počinju od 0.
- Proces može imati različit rang unutar različitih komunikatora.

```
int rank;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
printf("Moj rang unutar MPI_COMM_WORLD je %d\n", rank);
```

3.7 Dohvat informacija o komunikatoru: *ukupan broj procesa*

Proces može odrediti veličinu (ukupan broj procesa) svakog komunikatora kojem pripada pozivom funkcije `MPI_Comm_size()`.

Primjer:

```
int size;
MPI_Comm_size(MPI_COMM_WORLD, &size);
printf("Ukupan broj procesa u MPI_COMM_WORLD je %d\n", size);
```

Ako je komunikator `MPI_COMM_WORLD`, veličina koju vraća funkcija `MPI_Comm_size()` jednaka je broju definiranom prilikom pokretanja programa, primjerice:

```
mpirun -np 4 Program.out
```

Korištenjem varijabli okruženja:

```
export MP_PROCS=4
./Program.out
```

4 „Hello World!“ aplikacija

Primjer pokazuje verziju „Hello World“ programa, gdje svaki proces ispisuje svoj **rang** i **ukupan broj procesa** unutar komunikatora `MPI_COMM_WORLD`.

Primjer u C-u:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    printf("Pozdrav sa procesa %d od ukupno %d procesa!\n", rank, size);

    MPI_Finalize();
    return 0;
}
```

Primjer u C++-u (koristeći Boost.MPI:

```
#include <boost/mpi.hpp>
#include <iostream>

namespace mpi = boost::mpi;

int main() {
    mpi::environment env;
    mpi::communicator world;

    // Dohvat ranga i veličine komunikatora
    int world_rank = world.rank();
    int world_size = world.size();

    // Ispis podataka procesa
    std::cout << "Proces " << world_rank << " od " << world_size << ": Hello World!" <<
    std::endl;

    // Testiranje pogrešaka (opcionalno)
    if (world_rank == 0) {
        std::cout << "Sve rutine su uspješno izvršene na procesu 0." << std::endl;
    }

    // Automatsko upravljanje okruženjem kroz Boost.MPI
    return 0;
}
```

Analiza primjera

- Predefinirani komunikator: Oba primjera koriste MPI_COMM_WORLD, koji automatski obuhvaća sve procese unutar MPI programa.
- Sinkronizacija: Iako nije strogo potrebna u "Hello World" aplikaciji, MPI_Barrier se koristi kao primjer kako se procesi mogu sinkronizirati prije izlaza.
- Ispis rezultata: Redoslijed ispisa ovisi o brzini izvršenja procesa i nije deterministički bez dodatne sinkronizacije.
- Testiranje pogrešaka: Uvođenje testiranja pogrešaka, npr. provjerom povratnih vrijednosti funkcija, dodatno povećava robustnost programa.

Primjer rezultata

Ako se pokrene program na 4 procesa, izlaz bi mogao izgledati ovako:

```
Proces 2 od 4: Hello World!
Proces 1 od 4: Hello World!
Proces 3 od 4: Hello World!
Proces 0 od 4: Hello World!
Sve rutine su uspješno izvršene na procesu 0.
```

Svaki proces izvršava isti kod, uključujući ispitivanje ranga i veličine, te ispisa niza znakova (engl. string). Redoslijed ispisanih linija je nasumičan (prvi proces koji završi, prvi ispisuje liniju). Ne postoji unutrašnja sinkronizacija operacija između procesa. Svaki put kad se pokrene program može se promijeniti redoslijed linija. Dobra je praksa, ukoliko se želi ispis prema određenom redoslijedu, ubaciti kod koji regulira kad će koje linije biti ispisane (npr, funkcija `sleep(rang/100)`).

5 Izvori

- [1] <https://www.mpi-forum.org/docs/>
- [2] <https://www.open-mpi.org/doc/current/>
- [3] <https://www.codingame.com/playgrounds/349/introduction-to-mpi/introduction-to-distributed-computing>
- [4] <https://mpitutorial.com/>

Zadaci:

1. Napišite program koristeći neku od implementacija MPI-ja u kojem proces ranga 1 ispisuje informaciju o tome na koliko je procesa pokrenut MPI program. Pokrenite program na 1, 2 i 4 procesa i komentirajte rezultat.
2. Napišite program koristeći neku od implementacija MPI-ja koji radi sljedeće:
 - a. Otvaranje i pisanje u zajedničku datoteku:
 - Svi procesi otvaraju istu zajedničku datoteku (npr. rezultati.txt), ali osiguravaju da zapisuju podatke jedan po jedan koristeći sinkronizaciju.
 - b. Zapis podataka u datoteku:
 - Svaki proces zapisuje sljedeće podatke:
 1. Svoj rang (rank),
 2. Ukupan broj procesa (size),
 3. Naziv procesora na kojem se izvršava proces,
 - c. Oblik zapisa u datoteku:

P[rang] od [velicina] se izvršava na procesoru [naziv_procesora]. Prethodni proces napravio je [broj_zapisa] zapisa.

3. (NAPREDNA VERZIJA ZADATKA) Napišite program iz zadatka 2. na način da prvo napravite 2 dodatna komunikatora, u svakom od kojih se trebaju nalaziti po dva procesa. Procesi uz informacije koje ispisuju prema zadatku 2. trebaju ispisati i rang unutar novodefiniranog komunikatora.